EECS 545 Machine Learning
Progress Report
GNN on Traveling Salesman Problem

Pingbang Hu
Department of Computer Science
University of Michigan
pbb@umich.edu

Jonathan Moore
Department of Computer Science
University of Michigan
moorejon@umich.edu

Yi Zhou
Department of Math
University of Michigan
yizho@umich.edu

Shubham Kumar Pandey
Department of Statistics
University of Michigan
pandeysk@umich.edu

Anuraag Ramesh
Department of Statistics
University of Michigan
anuraagr@umich.edu

Abstract

This is our progress report for the EECS 545 course project. Graph neural network is a rising concept in computer science that has demonstrated significant potential in many areas. Our project aims to further demonstrate its potential by applying it to the classic traveling salesman problem. After generating the dataset, we will first use imitation learning to obtain a pretty good path, which will then be inputted into reinforcement learning to obtain the optimal solution. Since imitation learning converges very fast, we expect our run time to be promising, for larger datasets, compared to tradition exact solvers.

In this progress report, we will describe and explain our proposed method, share important related works, show our difference with the existing works, present our experimental results, and set up weekly goals for future development.

Keywords: Travelling salesman problem, Graph Neural Network, Imitation Training, Reinforcement Learning, Transformer, Integer Programming, Embedding learning, Link Prediction, Combinatorial Optimization, Learning theory.

1 Introduction

The travelling salesman problem, also called the travelling salesperson problem or TSP, is a problem as follows. Given a list of cities and the distances between each pair of cities, we want to find the shortest possible route that visits each city *exactly once* and returns to the origin city.

Specifically, given an **undirected weighted graph** $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, which is an ordered pair of nodes set $\mathcal{E}$ and edges set $\mathcal{V} \subseteq \mathcal{E} \times \mathcal{E}$ where $\mathcal{G}$ is equipped with **spatial structure**. This means that each edge between nodes will have different weights and each node will have its coordinates, we want to find a simple cycle that visits every node exactly once while having the smallest cost.

Our approach is based on the application of graph neural networks. After generating the dataset, we will first use imitation learning to obtain a pretty good path, which will then be inputted into unsupervised reinforcement learning to obtain the optimal solution. Since imitation learning converges very fast, we expect our run time to be promising, for large datasets, compared to tradition exact solvers.

2 Proposed Method

2.1 Problem Formulation

2.1.1 TSP as Integer Linear Programming

We first formulate TSP in terms of **Integer Linear Programming**. Given an undirected weighted group $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, we label the nodes with numbers $1, \ldots, n$ and define

$$x_{ij} := \begin{cases} 1, & \text{if } (i,j) \in \mathcal{E}' \\ 0, & \text{if } (i,j) \in \mathcal{E} \setminus \mathcal{E}', \end{cases}$$

where $\mathcal{E}' \subset \mathcal{E}$ is a variable which can be viewed as a compact representation of all variables $x_{ij}, \forall i, j$. Furthermore, we denote the weight on edge (i,j) by c_{ij}, then for a particular TSP problem instance, we can formulate the problem as follows.

$$\begin{aligned} \min & \sum_{i=1}^{n} \sum_{j \neq i, j=1}^{n} c_{ij} x_{ij} : \\ & x_{ij} \in \{0,1\} && i,j = 1, \ldots, n; \\ & u_i \in \mathbb{Z} && i = 2, \ldots, n; \\ & \sum_{i=1, i \neq j}^{n} x_{ij} = 1 && j = 1, \ldots, n; \\ & \sum_{j=1, j \neq i}^{n} x_{ij} = 1 && i = 1, \ldots, n; \\ & u_i - u_j + n x_{ij} \leq n - 1 && 2 \leq i \neq j \leq n; \\ & 1 \leq u_i \leq n - 1 && 2 \leq i \leq n. \end{aligned} \tag{1}$$

This is the Miller-Tucker-Zemlin formulation[11]. Note that in our case, since we're going to solve TSP exactly, hence all variables are integers, and we sometimes call this kind of integer linear programming as **pure integer programming**.

Since integer programming is a NP-Hard problem, there are no known polynomial algorithm can solve this explicitly. Hence, modern approach to such a problem is to **relax** the integrality constraint, which makes Equation 1 becomes a continuous linear programming (LP), whose solution provides a lower bound to Equation 1 since it's a relaxation, and we're trying to find the minimum.

Since an LP is a convex optimization problem, we have many polynomial time algorithms to solve the relaxed version. After obtaining a relaxed solution, if such LP relaxed solution respects the integrality constraint, we see that it's indeed a solution to Equation 1. But if not, we can simply divide the original relaxed LP into two sub-problems by **splitting the feasible region** according to a variable that does not respect integrality in the current relaxed LP solution $\boldsymbol{x}^*$,

$$x_i \leq \lfloor x_i^* \rfloor \vee x_i \geq \lceil x_i^* \rceil, \qquad \exists i \leq p \mid x_i^* \notin \mathbb{Z}. \tag{2}$$

We see that by adding such additional constraint in two sub-problems respectively, we get a recursive algorithm called **Branch-and-Bound** [15].

2.1.2 Branching Rules

The branch-and-bound algorithm is widely used to solve integer programming problems. We see that the key step in the branch-and-bound algorithm is selecting a non-integer variable to branch on in Equation 2. And as one can expect, some choices may reduce the recursive searching tree significantly [2], hence the *branching rules* are the core of modern combinatorial optimization solvers, and it has been the focus of extensive research [10, 12, 6, 1]. There are several popular strategies [3] used in modern solver.

1. Strong branching. It'll result in the smallest recursive tree by computing the expected bound improvement for **each** candidate variable before branching by finding solutions of two LPs for every candidate, which is undoubtedly, very expensive. [4]

2. Hybrid branching. Which basically computes a strong branching scores only at the beginning of the solving, and gradually switches to other methods like Conflict score, Pseudo-cost [12], or some hand-crafted combinations of above. [3, 1].

2.2 Theoretical Framework

Our objective is to learn branching strategy without expensive evaluation. And since this is a discrete time control process, hence we model the problem by Markov Decision Process (MDP) [8].

2.2.1 Markov Decision Process (MDP)

Given a regular Markov decision process $\mathcal{M} \coloneqq (\mathcal{S}, \mathcal{A}, p_{\text{init}}, p_{\text{trans}}, R)$, where we have

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Initial state distribution $p_{\text{init}} \colon \mathcal{S} \to \mathbb{R}_{\geq 0}$
- State transition distribution $p_{\text{trans}} \colon \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}_{\geq 0}$
- Reward function $R \colon \mathcal{S} \to \mathbb{R}$

We note that in the above definition of Markov Decision Process, the reward function R need not be deterministic. In other words, we can define R as a random function which will take a value based on a particular state in $\mathcal{S}$ with some randomness. Note that if R in $\mathcal{M}$ is equipped with any kind of randomness, we can write the reward r_t at time t as

$$r_t \sim p_{\text{reward}}(r_t \mid s_{t-1}, a_{t-1}, s_t),$$

and this can be converted into an equivalent Markov Decision Process $\mathcal{M}'$ with a deterministic reward function R', where the randomness is integrated into parts of the states.

With an action policy $\pi \colon \mathcal{A} \times \mathcal{S} \to \mathbb{R}_{\geq 0}$ such that the action a_t taken at time t is determined by

$$a_t \sim \pi(a_t \mid s_t),$$

we see that an MDP can be unrolled to produce a *trajectories* composed by state-action pairs as

$$\tau = (s_0, a_0, s_1, a_1, \ldots)$$

which obeys the joint distribution

$$\tau \sim \underbrace{p_{\text{init}}(s_0)}_{\text{initial state}} \prod_{t=0}^{\infty} \underbrace{\pi(a_t \mid s_t)}_{\text{next action}} \underbrace{p_{\text{trans}}(s_{t+1} \mid a_t, s_t)}_{\text{next state}}.$$

2.2.2 Partially Observable Markov Decision Process (PO-MDP)

Follows the same idea from MDP, in the PO-MDP setting deals with the case that when the **complete** information about the current MDP state $\mathcal{S}$ is unavailable or not necessarily for the decision-making [16]. Instead, in our case, only a partial **observation** $o \in \Omega$ is available, where Ω is called the **partial state space**. We can use an active perspective to view the above model, namely we're merely applying a observation function $O \colon \mathcal{S} \to \Omega$ to the current state s_t at each time step t. Hence, we define a PO-MDP $\widetilde{\mathcal{M}}$ as a tuple

$$\widetilde{\mathcal{M}} \coloneqq (\mathcal{S}, \mathcal{A}, p_{\text{init}}, p_{\text{trans}}, R, O).$$

Within this setup, a trajectory of PO-MDP takes form as

$$\tau = (o_0, r_0, a_0, o_1, r_1, a_1, \ldots),$$

where $o_t \coloneqq O(s_t)$ and $r_t \coloneqq R(s_t)$. Specifically, noting that here r_t still depends on the state of the OP-MDP, not the observation.

We introduce a convenience variable $h_t \colon (o_0, r_0, a_0, \ldots, o_t, r_t) \in \mathcal{H}$, which represents the PO-MDP history at time step t **without the action** a_t. Due to the non-Markovian nature of the trajectories,

$$o_{t+1}, r_{t+1} \not\perp h_{t-1} \mid o_t, r_t, a_t,$$

namely the decision-maker must take the whole history of observations, rewards and actions into account to decide on an optimal action at current time step t. We then see that action policy for PO-MDP takes the form $\widetilde{\pi} \colon \mathcal{A} \times \mathcal{H} \to \mathbb{R}_{\geq 0}$ such that $a_t \sim \pi(a_t \mid h_t)$.

2.3 Markov Control Problem

We define the MDP control problem as that of finding a policy $\pi^* : \mathcal{A} \times \mathcal{S} \to \mathbb{R}_{\geq 0}$ which is optimal with respect to the expected total reward, that is,

$$\pi^* = \arg\max_{\pi} \lim_{T \to \infty} \mathbb{E}_{\tau} \left[\sum_{t=0}^{T} r_t \right],$$

where $r_t := R(s_t)$. To generalize this into PO-MDP control problem, similar to the MDP control problem, the objective is to find a policy $\widetilde{\pi}^* : \mathcal{A} \times \mathcal{H} \to \mathbb{R}_{\geq 0}$ such that it maximizes the expected total rewards. By slightly abusing the notation, we simply denote this learned policy by $\widetilde{\pi}^*$ where the objective function is completely the same as in the MDP case.

2.4 Learning Pipeline

Since the branch-and-bound variable selection problem can be naturally formulated as a Markov decision process, hence a natural machine learning algorithm to use is reinforcement learning [14]. Specifically, since there are some state-of-the-art (SOTA) integers programming solvers out there, for examples, `Gurobi`[1], `SCIP`[2], etc, we decide to try imitating learning[9] by learning directly from an expert branching rule.

While there are some related works in this approach [7] aiming to tackle **mixed integer linear programming** (MILP), which means there are only some variables have integrality constraints, while other variables can be real numbers. But our approach tries to extend this further, we're focusing on TSP, which not only is a pure integer programming, and the variables value is bounded to be in $\{0, 1\}$ only. Except all these, since imitating learning can't outperform SOTA solver in its nature, hence though the work in [7] is interesting, but we still think we can do better by focusing on a particular domain (TSP in this case) and try to formulate a problem specific reward function for TSP.

Hence, our proposed learning pipeline is as follows. Firstly, we only use imitating learning following [7] to obtain a good enough model, then we turn to unsupervised learning setup under the framework of reinforcement learning, which should make the network learn some domain-specific property and help to boost the performance. The implementation summary can be found in Appendix A.1.

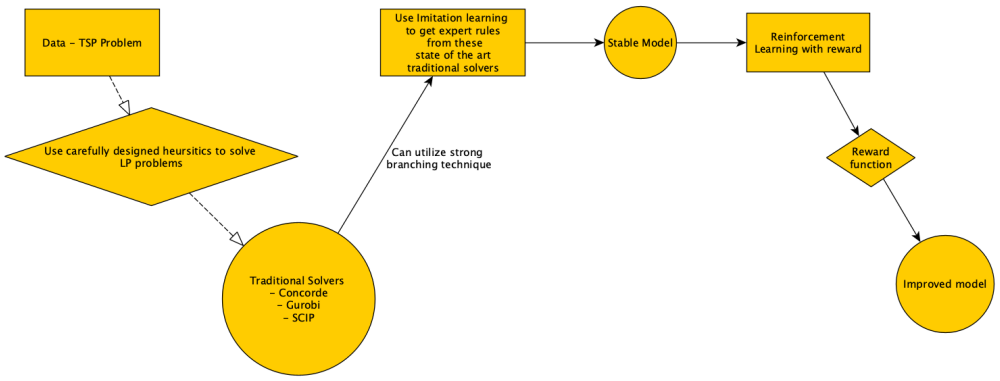

Figure 1: Implementation Pipeline

2.4.1 Imitating Learning

As we mentioned before, the most effective branching strategy is strong branching, hence we train the network by behavioral cloning [13]. We first run the SOTA solver and get state-action pairs

$$\mathcal{D} = \{(s_i, \boldsymbol{a}_i^*)\}_{i=1}^{N},$$

[1] https://www.gurobi.com/
[2] https://www.scipopt.org/

and then learn our policy $\widetilde{\pi}^*$ b minimizing the cross-entropy loss

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{(\boldsymbol{s},\boldsymbol{a}^*)\in\mathcal{D}} \log \widetilde{\pi}_\theta(\boldsymbol{a}^* \mid \boldsymbol{s}).$$

Additionally, we access (observe) the state of the branch-and-bound process by directly interacting with the SOTA solvers, specifically, `SCIP`. The implementation detail follows Gasse et al.[7].

2.4.2 Unsupervised Reinforcement Learning

After getting a stable model, we remove the expert's instructions and let the model explore by itself since we're focusing on TSP, which is a particular problem type in pure integer programming. That is, we are now going to define a new reward function $R\colon \mathcal{S} \to \mathbb{R}$.

We are still exploring this part, but some useful and observable states can be used in our formulation of R, including things like estimated branching tree sizes and its decreasing rate, or current tour length and its decreasing rate, etc.

And also, we have some interesting idea about the objective of such reward function, like reward decay forcing fast solving, or providing additional information, like the optimal tour length.

In this way, from the success in [7], we're guaranteed with a good enough model status, namely a good enough branching strategy after our first stage. Then, we do not need to worry about the technical detail in reinforcement learning like initial training but focus on giving a general pipeline for how to tackle a particular problem in this kind of combinatorial optimization problems.

Besides, there is a reason why we do not use pre-trained models from [7]. An obvious reason is that our focus is different: [7] focuses on general MILP, while we specifically focus on TSP (Pure IP with bounded variable values), which is a slightly but still different field.[3]

3 Related Work

Obviously, there has already been extensive work done to optimize TSP solvers both theoretically and practically. We have done extensive research into other solvers, the most important of which will be summarized.

Transformer Network for TSP [5] The main focus of this paper is to detail the application of deep reinforcement learning reapplied to a Transformer architecture originally created for NLP. Unlike our proposed model, this solver learns heuristics with very low error rates (.004% for 50 points and 0.39% for 100 points) which can run over a TSP problem much faster than a traditional solver with similar results.

Exact Combinatorial Optimization with GCNNs [7] This paper serves as the one of the backbones of our research; its main focus is to detail how MIPS can potentially be solved much quicker than a traditional solver by using GNNs (specifically GCNNs). It did this by training its model using imitation learning (using the strong branching expert rule) and was able to effectively produce outputs for problem instance much greater than what they were trained on. By using this method in our own model, we can train using the 1,000 node samples that traditional solvers can compute quickly while still being able to solve several thousand node examples ourselves.

State of the Art Exact Solver There has been a lot of progress on the symmetric TSP in the last century. With the increase in the number of nodes, there is a superpolynomial (at least exponential) explosion in the number of potential solutions. This makes the TSP problem difficult to solve on two parameters, the first being finding a global shortest route as well as reducing the computation complexity in finding this route. Concorde[4], written in the ANSI C programming language, is widely recognized as the state-of-the-art(fastest) exact TSP solution for large datasets.

[3] Using the pre-trained models will be a good idea to see the generalizability in RL after we obtain a good enough reward function.

[4] http://www.math.uwaterloo.ca/tsp/concorde/

Our Difference With Existing Work Our approach to solve TSP exactly is different from the Concorde approach, as well as the methods mentioned in the papers above. We will first use imitation training, which has been proven to converge fast, to obtain an approximation of the optimal path. Then, we will use our current path to initialize the reinforcement learning in order to obtain the global shortest path. Our combination of the two methods on GNN might be a novel appraoch in solving TSP exactly. For problems with more than 1000 nodes, we expect our approach to give promising run time compared to other existing exact solvers.

4 Experimental Results

Our implementation on the first part, imitating learning, is based on Ecole[5] and hugely follows the work by Gasse et al.'[7] We now have a very simple implementation similar to the setup in the original paper, but with cleaner API and code style.

For our data, we will be using a data generator created from the Concorde TSP solver. This data generator will allow us to efficiently generate TSP problem sets of any size that we want; additionally, we can use their solver as well in order to first generate our training set for imitation learning. Once we have moved out of imitation learning, it will still be very useful to be able to have the ability to (relatively) quickly check our model's output for correctness.

Fortunately, as long as we design our model around the output of the Concorde TSP generator, we do not need any further pre-processing. The result is included in Appendix A.

5 Future Milestone

1. Define an explicit reward function.
2. Using more sophisticated model in the imitating learning part.
3. Benchmarking all results from different exact solver.
4. Comparing the performance between using pre-trained model and not using one.
5. Implement the code for TSP data generator and also all the learning algorithms.
6. Give theoretical analysis of our result.

6 Conclusion

We have done a comprehensive and detailed research about this topic, and decide to turn our focus on approximation algorithm to exact algorithm for TSP compare to what we have proposed at the beginning. We explored the potential of combinatorial optimization problem in machine learning and tried to understand the previous works people have done. Additionally, we found the excellent library, Ecole, for the combinatorial optimization problem in machine learning and tried to learn it in detail and integrate it into our implementation.

Besides all the exploration works, we also looked into the mathematical rigorous framework to model our problem, and give a quick overview of why our methods should work based on the mathematical framework we gave. We are now ready to implement our proposed learning algorithm based on our current data generator moving forward.

7 Author Contributions

All authors contributed equally to this work. Yi and Jonathan worked on drafting the abstract, introduction and related work. Pingbang and Shubham worked on the proposed method, finding rationale for why it should work, and intuition behind choosing this specific approach. Pingbang drafted the proposed method including the technical details. Anuraag and Pingbang worked on the implementation part, including getting familiar with the Ecole library. Anuraag worked on instance and data set generation and running initial steps on this library. Jonathan drafted the data set generation and experimental results.

[5] https://github.com/ds4dm/ecole

References

[1] Tobias Achterberg and Timo Berthold. Hybrid branching. pages 309–311, 05 2009. ISBN 978-3-642-01928-9. doi: 10.1007/978-3-642-01929-6_23.

[2] Tobias Achterberg and Roland Wunderling. *Mixed Integer Programming: Analyzing 12 Years of Progress*, pages 449–481. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013. ISBN 978-3-642-38189-8. doi: 10.1007/978-3-642-38189-8_18. URL https://doi.org/10.1007/978-3-642-38189-8_18.

[3] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005. ISSN 0167-6377. doi: https://doi.org/10.1016/j.orl.2004.04.002. URL https://www.sciencedirect.com/science/article/pii/S0167637704000501.

[4] D. Applegate, R. Bixby, V. Chvatal, and B. Cook. Finding cuts in the tsp (a preliminary report). Technical report, 1995.

[5] Xavier Bresson and Thomas Laurent. The transformer network for the traveling salesman problem. *ArXiv*, abs/2103.03012, 2021.

[6] Matteo Fischetti and Michele Monaci. Branching on nonchimerical fractionalities. *Operations Research Letters*, 40:159–164, 05 2012. doi: 10.1016/j.orl.2012.01.008.

[7] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32*, 2019.

[8] R.A. Howard. *Dynamic programming and Markov processes*. Technology Press of Massachusetts Institute of Technology, 1960. URL https://books.google.com/books?id=fXJEAAAAIAAJ.

[9] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. Imitation learning: A survey of learning methods. *ACM Comput. Surv.*, 50(2), apr 2017. ISSN 0360-0300. doi: 10.1145/3054912. URL https://doi.org/10.1145/3054912.

[10] Jeff T. Linderoth and Martin W. P. Savelsbergh. A computational study of search strategies for mixed integer programming. *INFORMS J. Comput.*, 11(2):173–187, 1999. doi: 10.1287/ijoc.11.2.173. URL https://doi.org/10.1287/ijoc.11.2.173.

[11] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *J. ACM*, 7(4):326–329, oct 1960. ISSN 0004-5411. doi: 10.1145/321043.321046. URL https://doi.org/10.1145/321043.321046.

[12] Jagat Patel and John Chinneck. Active-constraint variable ordering for faster feasibility of mixed integer linear programs. *Math. Program.*, 110:445–474, 09 2007. doi: 10.1007/s10107-006-0009-0.

[13] Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991. doi: 10.1162/neco.1991.3.1.88.

[14] R.S. Sutton and A.G. Barto. *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series. MIT Press, 2018. ISBN 9780262352703. URL https://books.google.com/books?id=uWV0DwAAQBAJ.

[15] Laurence Wolsey. *Branch and Bound*, chapter 7, pages 113–138. John Wiley and Sons, Ltd, 2020. ISBN 9781119606475. doi: https://doi.org/10.1002/9781119606475.ch7. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119606475.ch7.

[16] K.J Åström. Optimal control of markov processes with incomplete state information. *Journal of Mathematical Analysis and Applications*, 10(1):174–205, 1965. ISSN 0022-247X. doi: https://doi.org/10.1016/0022-247X(65)90154-X. URL https://www.sciencedirect.com/science/article/pii/0022247X6590154X.

Appendix

A Sample Code Output

A.1 General Pipeline

Our code will include

- `01_generate_instances.py`.
- `02_generate_dataset.py`.
- `03_train_gnn.py`.
- `04_evaluate.py`.

We report the result of `01_generate_instances.py` and `02_generate_dataset.py` and leave the output for `03_train_gnn.py` and `04_evaluate.py` to the next time.

The overall pipeline should be summarized to the following simple script.

```
# Generate MILP instances
python 01_generate_instances.py TSP

# Generate supervised learning datasets
python 02_generate_dataset.py TSP -j 4  # number of available CPUs

# Training
for i in {0..4}
do
    python 03_train_gnn.py TSP -s $i
done

# Evaluation
python 04_evaluate.py TSP
```

A.2 Data Generation

In this section, we report some sample output after running our implementation for problem instances generator. Specifically, we implement a dataset generator which will do the followings in sequence.

1. `01_generate_instances.py`. This will essentially do
   (a) Generate instances of problem randomly by the number we specify.
   (b) Pass the generated problem instances to `SCIP` and record all the observation from `SCIP` during the solving.
2. `02_generate_dataset.py`. This will essentially do
   (a) Wrap problem instances and also the corresponding observations into one big dataset to use later.

A.2.1 Instance Generation

After utilizing the Ecole library, we have tried to generate some sample problem instances for `setcover`. Here is the result.

```
1000 instances in data/instances/setcover/train_500r_1000c_0.05d
200 instances in data/instances/setcover/valid_500r_1000c_0.05d
10 instances in data/instances/setcover/transfer_50r_1000c_0.05d
10 instances in data/instances/setcover/transfer_100r_1000c_0.05d
10 instances in data/instances/setcover/transfer_200r_1000c_0.05d
200 instances in data/instances/setcover/test_50r_100c_0.05d
```

```
  generating file data/instances/setcover/train_500r_1000c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/train_500r_1000c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/train_500r_1000c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/train_500r_1000c_0.05d/instance_999.lp ...
  generating file data/instances/setcover/train_500r_1000c_0.05d/instance_1000.lp ...
  generating file data/instances/setcover/valid_500r_1000c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/valid_500r_1000c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/valid_500r_1000c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/valid_500r_1000c_0.05d/instance_199.lp ...
  generating file data/instances/setcover/valid_500r_1000c_0.05d/instance_200.lp ...
  generating file data/instances/setcover/transfer_50r_1000c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/transfer_50r_1000c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/transfer_50r_1000c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/transfer_50r_1000c_0.05d/instance_9.lp ...
  generating file data/instances/setcover/transfer_50r_1000c_0.05d/instance_10.lp ...
  generating file data/instances/setcover/transfer_100r_1000c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/transfer_100r_1000c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/transfer_100r_1000c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/transfer_100r_1000c_0.05d/instance_9.lp ...
  generating file data/instances/setcover/transfer_100r_1000c_0.05d/instance_10.lp ...
  generating file data/instances/setcover/transfer_200r_1000c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/transfer_200r_1000c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/transfer_200r_1000c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/transfer_200r_1000c_0.05d/instance_9.lp ...
  generating file data/instances/setcover/transfer_200r_1000c_0.05d/instance_10.lp ...
  generating file data/instances/setcover/test_50r_100c_0.05d/instance_1.lp ...
  generating file data/instances/setcover/test_50r_100c_0.05d/instance_2.lp ...
  generating file data/instances/setcover/test_50r_100c_0.05d/instance_3.lp ...
  ...
  generating file data/instances/setcover/test_50r_100c_0.05d/instance_199.lp ...
  generating file data/instances/setcover/test_50r_100c_0.05d/instance_200.lp ...
done.
```

A.2.2 Dataset Generation

Also, we have implement the code template which split the data into training set, validation and also testing set and wrapped them up. Here is the output.

```
seed 0
1000 train instances for 1000 samples
200 validation instances for 200 samples
0 test instances for 200 samples
[w Thread-2] episode 0, seed 3789440165, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_154.lp'...
[w Thread-3] episode 1, seed 3998999672, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_651.lp'...
[w Thread-4] episode 2, seed 3665041058, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_364.lp'...
[w Thread-5] episode 3, seed 2122579412, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_829.lp'...
[w Thread-4] episode 2 done, 3 samples
[w Thread-4] episode 4, seed 92199959, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_706.lp'...
[m MainThread] 1 / 1000 samples written, ep 0 (4 in buffer).
[m MainThread] 2 / 1000 samples written, ep 0 (4 in buffer).
[m MainThread] 3 / 1000 samples written, ep 0 (4 in buffer).
...
[m MainThread] 111 / 1000 samples written, ep 0 (226 in buffer).
[m MainThread] 112 / 1000 samples written, ep 0 (231 in buffer).
[m MainThread] 113 / 1000 samples written, ep 0 (233 in buffer).
[w Thread-2] episode 0 done, 113 samples
[w Thread-2] episode 5, seed 3084161443, processing instance 'data/instances/setcover/
    train_500r_1000c_0.05d/instance_492.lp'...
...
[m MainThread] 999 / 1000 samples written, ep 1 (2899 in buffer).
[m MainThread] 1000 / 1000 samples written, ep 1 (2901 in buffer).
[w Thread-7] episode 0, seed 4086331612, processing instance 'data/instances/setcover/
    valid_500r_1000c_0.05d/instance_116.lp'...
[w Thread-8] episode 1, seed 4177056010, processing instance 'data/instances/setcover/
    valid_500r_1000c_0.05d/instance_34.lp'...
[w Thread-9] episode 2, seed 1004525138, processing instance 'data/instances/setcover/
    valid_500r_1000c_0.05d/instance_36.lp'...
[w Thread-10] episode 3, seed 2483450156, processing instance 'data/instances/setcover/
    valid_500r_1000c_0.05d/instance_14.lp'...
```

```
[w Thread-4] episode 14 done, 1078 samples
[m MainThread] 1 / 200 samples written, ep 0 (5 in buffer).
[m MainThread] 2 / 200 samples written, ep 0 (8 in buffer).
[m MainThread] 3 / 200 samples written, ep 0 (12 in buffer).
...
```